به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴)

دانشگاه صنعتی شریف — تمرین دوم

مدرس: مهدی صفرنژاد — انتشار: ۲۲ مهر ۹۸

مرتب‌سازی

زمان تحویل این تمرین (شامل بخش نظری و بخش عملی) شنبه ۴ آبان ۹۸ ساعت ۱:۰۰ بامداد (یک ساعت پس از نیمه‌شب جمعه) می‌باشد. به توضیحات سایت درس دربارهٔ تمرین‌ها، تحویل آن و تأخیر توجه کنید. تمرین‌ها از سایت کوئرا تحویل گرفته می‌شوند. قسمت‌های نظری به صورت فایل PDF تحویل گرفته می‌شوند که این فایل یا باید با حروف‌چینی رایانه‌ای تولید شده باشد و یا اگر دست‌نوشته است، توسط نرم‌افزارهایی مثل CamScanner یا Office Lens و یا با دستگاه اسکنر تصویربرداری شده باشد.

سؤال‌های ۱ تا ۳ تئوری هستند و برای سؤال‌های ۴ تا ۷ باید برنامه‌تان را در در کوئرا ارسال کنید.

سؤال ۱. [۱۲ نمره] ثابت کنید مرتبه زمانی الگوریتم مرتب سازی درجی به صورت میانگین $\Theta(n^۲)$ است.

پاسخ:

لم: تعداد جابجایی ها در مرتب سازی درجی برابر است با:

$$T(n) = \sum_{i=۰}^{n-۱} max(۰, i - A[i])$$

که $A[i]$ جایگاه عضو i ام آرایه در آرایه مرتب شده است. اثبات لم براحتی با استقرا روی n قابل انجام است.

حال به اثبات مسئله می پردازیم:

طبق لم داریم:

$$E(T(n)) = \sum_{i=۰}^{n-۱} E(max(۰, i - A[i]))$$

چون $A[i]$ هر از یک از مقادیر ۰ تا $n-۱$ را با احتمال مساوی اخذ می کند پس داریم :

$$E(max(۰, i - A[i])) = \sum_{j=۰}^{n-۱} \frac{۱}{n} max(۰, i - j) = \frac{۱}{n} \sum_{k=۱}^{i} k = \frac{i(i+۱)}{۲n}$$

پس داریم :

$$E(T(n)) = \frac{۱}{۲n} \sum_{i=۰}^{n-۱} (i^۲ + i) = \frac{(n+۱)(۲n+۱)}{۱۲} + \frac{n+۱}{۴} = \theta(n^۲)$$

سؤال ۲. [۱۲ نمره] به سؤالات زیر در مورد هرم کمینه پاسخ دهید:

الف) فرض کنید اعداد دنباله‌ی زیر را به ترتیب از چپ به راست اضافه می‌کنیم. تعداد مراحل جابه‌جایی مقادیر روی رئوس درخت چند تا خواهد بود؟

$$< ۲, ۵, ۴, ۳, ۱ >$$

ب) فرض کنید اعداد n تا ۱ را به ترتیب در درخت اضافه می‌کنیم. در این صورت چندبار عمل جابه‌جایی مقادیر روی رئوس درخت‌مان انجام خواهد شد؟

پاسخ: الف) اضافه کردن ۲ و ۵ و ۴ به هیچ جابه‌جایی‌ای نیاز ندارد. اما با اضافه کردن ۳، دو جابه‌جایی با بالایی‌هایش انجام می‌دهیم با اضافه کردن ۱ نیز همین‌طور باید با پدرهایش جابه‌جا شود تا ریشه قرار گیرد. در نتیجه جواب این بخش $۴ = ۲ + ۲$ است.

ب) در هر مرحله عنصر جدیدی که اضافه می‌شود قرار است عنصر کمینه باشد بنابراین با تمامی پدرهایش تا ریشه جابه‌جا می‌شود. اگر ۲^k را بزرگترین توان ۲ای که کمتر اکید از n است در نظر بگیریم. به اندازه‌ی مقدار زیر طول می‌کشد.

$$(k+۱) \times (n - ۲^k) + k۲^k + (k-۲)۲^{k-۱} + ... + ۲^۱ = (k+۱) \times (n - ۲^k) + (k-۱) \times ۲^{k+۱} + ۲$$

$$k = \lfloor log_۲ n \rfloor$$

سؤال ۳. [۱۶ نمره]

در این مسئله می‌خواهیم مدل تعمیم یافتهٔ الگوریتم مرتب‌سازی ادغامی را بررسی کنیم. در این الگوریتم به‌جای اینکه آرایهٔ اولیه را به دو بخش تقسیم کنیم، آن را به k بخش تقسیم می‌کنیم؛ هر کدام از بخش‌ها را به صورت بازگشتی مرتب کرده و سپس k لیست مرتب شده را با هم ادغام می‌کنیم. برای ساده‌سازیِ تحلیل می‌توانید فرض کنید n توان کاملی از k است.

آ. [۴ نمره] با استفاده از داده‌ساختار هرم، الگوریتمی ارائه کنید که در زمان $O(n \log k)$ لیست‌ها را ادغام کرده و یک لیست مرتب nتایی تحویل دهد.

ب. [۸ نمره] حال فرض کنید الگوریتم شما فقط باید شامل تعدادی فراخوانی تابع $\text{merge}(a,\ b)$ باشد که دو لیست a و b را ورودی گرفته و ادغام شدهٔ آن‌ها را برمی‌گرداند. هزینهٔ فراخوانی این تابع $\Theta(|a| + |b|)$ است که $|a|$ و $|b|$ به ترتیب اندازهٔ لیست‌های a و b می‌باشند. مجدداً الگوریتمی ارائه دهید که در زمان $O(n \log k)$ لیست‌ها را ادغام کرده و یک لیست مرتب n تایی تحویل دهد.

پ. [۴ نمره] فرض کنید از روش دوم برای ادغام k لیست استفاده کنیم. رابطه بازگشتی زمان اجرای الگوریتم مرتب سازی ادغامی تعمیم یافته را بنویسید. سپس نشان دهید که زمان اجرای این الگوریتم (مستقل از مقدار k) حتما از $\Omega(n \log n)$ است.

پاسخ:

آ) برای اینکار کافیست ابتدا از روی k عضو اول لیست‌ها یک هرم کمینه بسازیم. راس‌های این هرم علاوه بر مقدار، یک اشاره‌گر به لیستی که از آن آمده‌اند نیز دارند. سپس در هر مرحله عضو کمینه هرم را استخراج کرده و به عنوان عضو بعدی لیست ادغام شده قرار می‌دهیم، سپس اگر عضوی از لیستی که عضو کمینه به آن تعلق داشت باقی‌مانده بود، عضو بعدی آن لیست را در هرم کمینه درج می‌کنیم. این کار را تا خالی شدن تمام لیست‌ها ادامه می‌دهیم. پیچیدگی زمانی ساخت این هرم $O(k)$ بوده و از آنجایی که هر عضو نیز دقیقا یک‌بار در هرم درج و حذف شده پیچیدگی زمانی کلی برابر است با $O(k + n\log k) = O(n\log k)$

ب) لیست‌ها را به $\lceil \frac{k}{۲} \rceil$ گروه حداکثر دوتایی تقسیم می‌کنیم و اعضای این گروه‌ها را ادغام می‌کنیم. در مرحله بعدی $\lceil \frac{k}{۲} \rceil$ لیست جدید داریم که مجدداً آن‌ها را دو به دو ادغام می‌کنیم و این کار را تا جایی ادامه می‌دهیم که تنها یک لیست باقی بماند. مراحل ادغام معادل با یک درخت دودویی پر هستند که برگ‌هایش لیست‌های اولیه می‌باشند. در هر عمقی از این درخت تعداد عملیات‌های انجام شده برای ادغام تمام لیست‌ها از $O(n)$ می‌باشد و با توجه به اینکه تعداد برگ‌ها k است حداکثر ارتفاع این درخت $\lceil \log k \rceil$ است. بنابراین پیچیدگی زمانی این الگوریتم نیز $O(n\log k)$ است.

پ) به سادگی می‌توان نشان داد که پیچیدگی زمانی روش دوم از $\Theta(n\log k)$ است. حال برای زمان اجرای مرتب‌سازی ادغامی تعمیم‌یافته داریم

$$T(n) = kT(\frac{n}{k}) + f(n)$$

و می‌دانیم

$$f(n) \in \Theta(n\log k) \rightarrow \exists C > ۰ \; f(n) \geq Cn\log k$$

می‌خواهیم به کمک استقرا برروی n ثابت کنیم

$$\exists A > ۰ \; T(n) \geq An\log n$$

بنابه فرض استقرا رابطه

$$T(x) \geq Ax\log x$$

برای تمام x های $k, k^۲, k^۳, ..., \frac{n}{k}$ برقرار است. آنگاه داریم

$$kT(\frac{n}{k}) \geq kA\frac{n}{k}\log\frac{n}{k} = An\log n - An\log k$$

$$T(n) \geq kT(\frac{n}{k}) + Cn\log k \geq An\log n - An\log k + Cn\log k$$

و اگر A را به گونه‌ای انتخاب کنیم که $A \leq C$ آنگاه خواهیم داشت. $T(n) \geq An\log n$

همچنین پایه استقرا نیز حالت $n = k$ است که در این حالت نیز داریم

$$T(k) \geq kT(۱) + Ck\log k \geq Ak\log k$$

بنابراین به کمک استقرا ثابت کردیم

$$\exists A > ۰ \; T(n) \geq An \log n$$

پس اثبات تمام است.

سؤال ۴. [۱۵ نمره]

k تا عدد $a_1, a_2, ..., a_k$ و n پیشنهاد داریم که هر پیشنهاد می‌تواند یکی از دو نوع زیر باشد:

۱. عدد b به a_i اضافه شود.

۲. عدد b در a_i ضرب شود.

باید m تا از این پیشنهادها را انتخاب کنید و به یک ترتیب دلخواه آن‌ها را اعمال کنید، به طوری که $\prod_{i=1}^{k} a_i$ ماکسیمم شود.

راهنمایی: برای مرتب کردن اعداد کسری مثل $\frac{a}{b}$ به طوری که a و b هر دو عدد صحیح هستند، اگر از متغیر float استفاده کنیم و دو عضو را مقایسه کنیم، امکان خطا وجود دارد، برای از بین بردن خطا می‌توان از این روش استفاده کرد: هر زمان به نتیجه $\frac{a}{b} \leq \frac{a'}{b'}$ نیاز داشتیم به جای آن از نتیجه $ab' \leq a'b$ استفاده می‌کنیم که با هم معادل هستند با این خاصیت که در نامساوی دوم هر دو طرف نامساوی عدد صحیح هستند.

ورودی:

در خط اول ورودی به‌ترتیب k، n و m داده می‌شود، در خط دوم ورودی اعداد $a_1, a_2, ..., a_k$ داده می شوند و در n خط بعدی پیشنهادها به صورت t_j، i_j و b_j داده می‌شود که t_j نوع پیشنهاد jاُم می‌باشد.

خروجی:

در تنها خط خروجی **باقیمانده** $\prod_{i=1}^{k} a_i$ را بر $10^9 + 7$ چاپ کنید (پس از اعمال m پیشنهادی که آن را ماکسیمم می‌کنند).

محدودیت‌ها:

$1 \leq k \leq 10^5$

$0 \leq m \leq n \leq 10^5$

$1 \leq a_i \leq 10^4$

$1 \leq t_j \leq 2$

$1 \leq i_j \leq k$

$1 \leq b_j \leq 10^4$

ورودی و خروجی نمونه:

stdin	stdout
5 4 1 2 2 5 2 5 1 5 5 1 1 3 2 1 2 2 5 2	500

پاسخ: پیشنهادهای نوع ۱ برای یک a_i خاص را در نظر بگیرید، اگر برحسب b_jهایشان از بزرگ به کوچک مرتب کنیم، حتما یک پیشوند از آن (که می‌تواند تهی باشد) انتخاب شده است (وگرنه می‌توان پیشنهادهای دیگری انتخاب کرد که ضرب اعداد بیشتر

شود)، فرض کنید بعد از مرتب کردن به $b'_1 \leq b'_2 \leq ... \leq b'_t$ رسیدیم در این صورت با توجه به چیزی که گفتیم می‌توان این پیشنهادهای نوع اول را به پیشنهادهای نوع دوم تبدیل کرد به این‌صورت که b درخواست jـ ام آن را برابر $\frac{a_i+\sum_{i=1}^{j} b'_i}{a_i+\sum_{i=1}^{j-1} b'_i}$ قرار می‌دهیم. پس از تبدیل کردن همه‌ی پیشنهادهای نوع ۱ به نوع ۲ با استفاده از روش بالا کافیست m پیشنهادی را انتخاب کنیم که b بیشتری دارند (چون هر پیشنهاد نوع ۲، $\prod_{i=1}^{k} a_i$ را در b خودش ضرب می‌کند).

سؤال ۵. [۱۵ نمره]

الی و جول در *آخرین ما*[1] از اکتشاف در دنیای آخرالزمانی‌شان خسته شده‌اند و تصمیم می‌گیرند بازی جدیدی برای وقت‌گذرانی انجام دهند.

آن‌ها برای این کار دنبالهٔ اعداد طبیعی دلخواه $a_1, a_2, \ldots, a_n$ را در نظر می‌گیرند. به ازای هر جفت a_i و a_j $(1 \le i < j \le n)$، اگر $a_i > a_j$ باشد الی $a_i + a_j$ امتیاز، اگر $a_i < a_j$، جول $a_i + a_j$ امتیاز و اگر $a_i = a_j$ هر کدام $(a_i + a_j)/2$ امتیاز می‌گیرند. کسی که در نهایت امتیاز بیشتری داشته باشد، برنده است.

پس از مدتی، آن‌ها از این بازی هم خسته می‌شوند و می‌خواهند بدانند چه کسی برنده می‌شود.

ورودی:

در خط اول ورودی عدد n داده می‌شود که تعداد اعداد دنباله است. در خط بعدی n عدد دنباله با فاصله از هم می‌آیند.

خروجی:

در خط اول خروجی بسته به نتیجهٔ بازی "Ellie"، "Joel" یا "draw" (بدون نقل قول)، و در خط دوم امتیاز برنده (در صورت تساوی امتیاز هر کدام) را نمایش دهید.

محدودیت‌ها:

$1 \le n \le 3 \times 10^5$

$1 \le a_i \le 10^8$

ورودی و خروجی نمونه:

stdin	stdout
3 7 1 1	Ellie 17

stdin	stdout
4 3 1 4 2	draw 15

stdin	stdout
5 1 2 3 4 5	Joel 60

پاسخ: اگر a_i و a_j برابر باشند نصف امتیاز، و اگر در دنباله نابجایی داشته باشند (یعنی $a_i > a_j$ در حالی که $i < j$)، کل امتیاز این جفت به الی می‌رسد. ساده‌ترین الگوریتم، برای هر جفت a_i و a_j امتیاز را محاسبه می‌کند. اما پیچیدگی زمانی آن از مرتبهٔ $O(n^2)$ است. با استفاده از ایده مرتب‌سازی ادغامی می‌توان الگوریتمی سریع‌تر با مرتبهٔ زمانی $O(n \log n)$ ارائه داد.

الگوریتم مرتب‌سازی ادغامی را برای دنباله اجرا کنید. مرحله ادغام را در این الگوریتم در نظر بگیرید. هر دو عدد a_i و a_j که نابجایی داشته باشند $(a_i > a_j$ و $i < j)$، دقیقا یک بار در دو دنبالهٔ مختلف قرار می‌گیرند که الگوریتم می‌خواهد ادغامشان کند. این دو

[1] The Last of Us

دنباله را $b_1 \leq b_2 \leq \cdots \leq b_m$ و $c_1 \leq c_2 \leq \cdots \leq c_k$ بگیرید که a_i در دنبالهٔ اول و a_j در دنبالهٔ دوم قرار دارد؛ یعنی tای وجود دارد که $c_t = a_j$. الگوریتم ادغام اشاره‌گرهایی را در ابتدای هر دو دنباله قرار می‌دهد و هر کدام را که عدد کوچکتری نشان می‌داد، جلو می‌برد. این اشاره‌گر پس از زمانی به c_t می‌رسد و c_t را که در دنبالهٔ اولیه قبل از هر کدام از اعداد $b_s, b_{s+1}, \ldots, b_m$ قرار داشت (s و t به ترتیب اشاره‌گرهای اول و دوم هستند)، جلوی آن‌ها قرار می‌دهد و نابجایی‌اش را با آن‌ها را از بین می‌برد. دقت کنید که کل الگوریتم مرتب‌سازی ادغامی صرفا متشکل از چندین مرحله ادغام دو زیر دنباله از دنباله اصلی است و هر جفت نابجایی دقیقا در یکی از این ادغام‌ها ترتیب‌شان عوض می‌شود. در هر مرحله ادغام، امتیازی که الی بدست می‌آورد را محاسبه می‌کنیم. یعنی وقتی اشاره‌گر دوم به c_t رسید، $(b_s + c_t) + (b_{s+1} + c_t) + \cdots + (b_m + c_t) = b_s + b_{s+1} + \cdots + b_m + (m - s + 1) \times c_t$ را به الی اضافه کنیم. برای اینکه این محاسبه، مرتبهٔ پیچیدگی زمانی ادغام را از $O(m + k)$ بیشتر نکند، می‌توانیم در ابتدای هر مرحله ادغام مجموع‌های جزئی $b_m, b_m + b_{m-1}, \ldots, b_m + b_{m-1} + \cdots + b_1$ را در جایی نگه داریم.

در آخر توجه کنید که مجموع امتیازهای الی و جول خیلی ساده به‌دست می‌آید و کافیست فقط امتیاز یکی را به‌دست بیاوریم.

سؤال ۶. [۱۵ نمره]

به تازگی سیروس مرتب‌سازی هرمی را پیاده‌سازی کرده است (برای مرتب کردن اعداد از بزرگ به کوچک، او از هرم کمینه استفاده می‌کند). به این ترتیب که n عدد از ورودی می‌خواند و آن‌ها را تک تک به وسیله عمل bubble-up داخل هرم درج می‌کند. سپس هربار کوچک‌ترین عضو هرم (یا همان عضو سر هرم) را با عضو آخر آن جابه‌جا می‌کند. سپس عضوی که هم‌اکنون سر هرم قرار گرفته است را توسط عمل bubble-down سر جای خود قرار می‌دهد. از آنجایی که او به پیچیدگی زمانی کدش شک دارد، می‌خواهد بداند در اجرای برنامه‌اش چند بار عمل جابه‌جا کردن انجام می‌شود.

ورودی:
در خط اول ورودی عدد n و در خط دوم یک آرایه از اعداد متمایز به شما داده می‌شود.

خروجی:
در تنها خط خروجی تعداد بارهای استفاده از عمل جابه‌جا کردن را چاپ کنید.

محدودیت‌ها:
$۱ \leq n \leq ۱۰^۵$
$۱ \leq a_i \leq ۱۰^۹$

ورودی و خروجی نمونه:

stdin	**stdout**
3 3 2 1	4

stdin	**stdout**
4 4 2 3 1	7

پاسخ: پاسخ نمونه.

سؤال ۷. [۱۵ نمره]

n ساختمان در کنار هم در یک ردیف قرار گرفته‌اند. می‌دانیم ارتفاع ساختمان i ام a_i است و عرض تمام ساختمان‌ها برابر ۱ است و ساختمان‌ها کاملا به هم چسبیده‌اند.

می‌خواهیم بنری روی این ساختمان‌ها نصب کنیم، به نحوی که شکل بنر باید به صورت مستطیل باشد و کاملا جلوی تعدادی از ساختمان‌ها قرار بگیرد (یعنی حتما پشتش کامل ساختمان باشد)

با گرفتن اطلاعات ساختمان‌ها بگویید حداکثر مساحتی که بنر می‌تواند داشته باشد چقدر است؟

راهنمایی: سعی کنید با استفاده از داده‌ساختار پشته، برای هر ساختمان اولین ساختمان سمت چپ و اولین ساختمان سمت راست آن که ارتفاع کمتری دارند را بیابید و با استفاده از این اطلاعات، مسئله را حل کنید.

ورودی:
درخط اول ورودی عدد n و طول ساختمان‌ها به شما داده شده‌است..

خروجی:
در تنها خط خروجی حداکثر مساحتی که بنر می‌تواند داشته باشد را چاپ کنید.

محدودیت‌ها:
$۱ \leq n \leq ۱۰^۵$

$۱ \leq a_i \leq ۱۰^۹$

ورودی و خروجی نمونه:

stdin	stdout
4 5 6 9 1	15

stdin	stdout
4 3 1 5 1	5

پاسخ: منطقی است اگر یک بازه از ساختمان‌ها را انتخاب کنیم که بنر در جلوی آن‌هاست، ارتفاع بنر را حداقل ارتفاع ساختمان‌های این بازه در نظر بگیریم.

در نتیجه فیکس می‌کنیم که کدام ساختمان حداقل ارتفاع بازه را قرار است داشته باشد، حداکثر طول این بازه این است که از سمت راست اولین ساختمان کوچک‌تر سمت چپ تا سمت چپ اولین ساختمان کوچک‌تر سمت راستش باشد، به اینصورت جواب را با این حالت ماکسیمم می‌گیریم.

پیروز باشید!